



Data Validation

Занятие 12 · Инструктор — **Андрей Помелов**



Что такое валидация данных?

Валидация данных — это процесс проверки входящих данных на корректность, полноту и соответствие ожидаемому формату *перед* их использованием в программе.

Входные данные

Данные поступают из форм, API, файлов или баз данных

Проверка

Программа анализирует структуру, тип и значения

Результат

Только корректные данные допускаются к обработке



Зачем нужна валидация?

Без проверки данных программа становится уязвимой к сбоям и непредсказуемому поведению.

→ **Предотвращение ошибок**

Некорректные данные не попадают в логику программы

→ **Стабильность работы**

Приложение ведёт себя предсказуемо при любых входных данных

→ **Качество данных**

Сохраняется целостность и достоверность информации

Проблемы без валидации

Когда данные не проверяются, возникают три типичных класса ошибок:



Отсутствующие поля

Обязательные данные не переданы — программа обращается к `undefined` или `null`



Неправильные типы

Строка вместо числа, массив вместо объекта — логика ломается на первой операции



Неожиданные значения

Отрицательная цена, пустое имя или неизвестный статус разрушают бизнес-логику

Пример: API списка продуктов

Представим, что приложение получает список товаров от внешнего API. Вот как могут выглядеть **проблемные данные**:

```
[  
  { "id": 1, "name": "Laptop", "price": 999, "category": "electronics" },  
  { "id": 2, "name": "", "price": -50, "category": "electronics" },  
  { "id": 3, "price": "free", "category": null },  
  { "id": 4, "name": "Mouse", "price": 25 }  
]
```

⚠ Запись 2

Пустое имя и отрицательная
цена

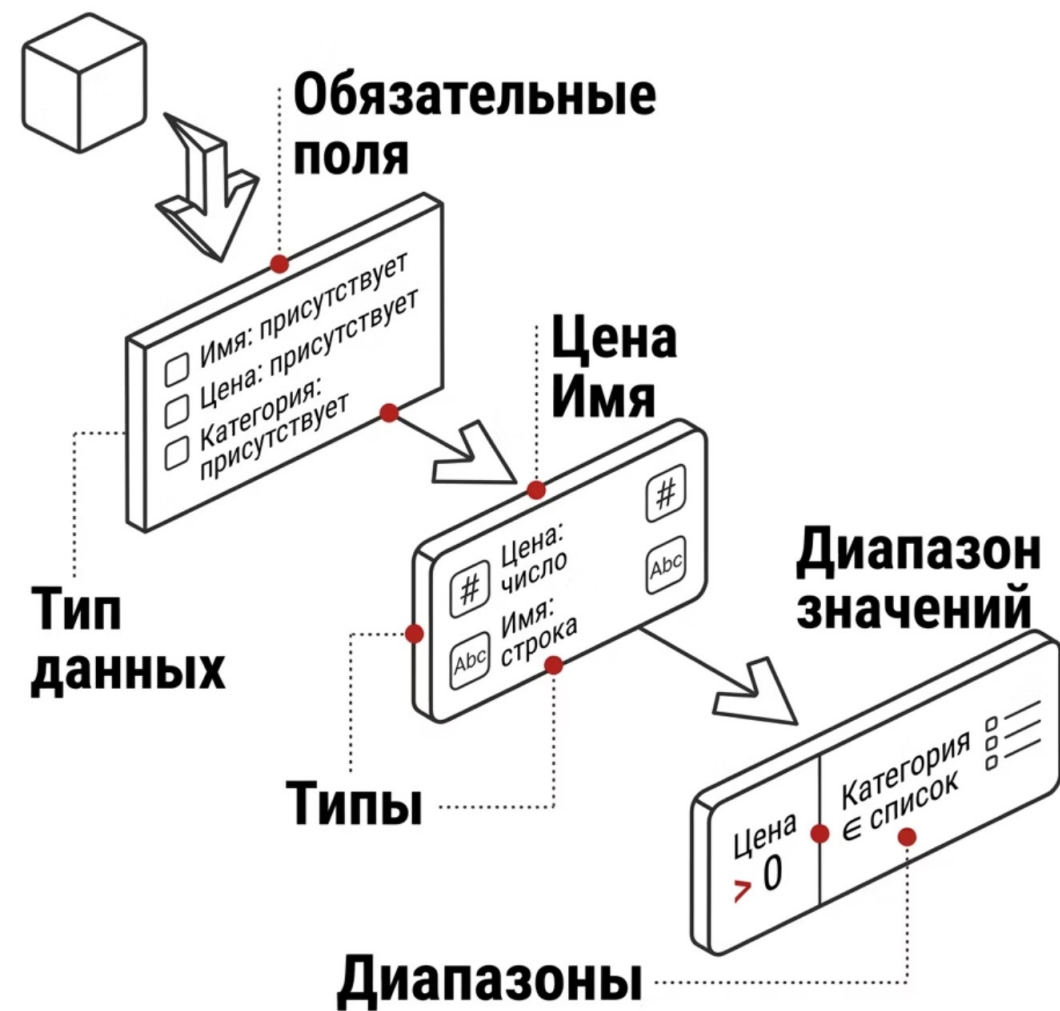
⚠ Запись 3

Нет поля `name`, цена — строка,
категория — `null`

⚠ Запись 4

Отсутствует поле `category`

Примеры проверок



Три уровня проверки

Наличие полей

Все обязательные ключи должны присутствовать в объекте

Тип данных

```
typeof price === "number", typeof name === "string"
```

Допустимые значения

Цена > 0, имя непустое, категория из разрешённого списка

Ручная логика валидации

Простейший способ валидации — использование условных операторов `if`:

```
function validateProduct(product) {  
  if (!product.name || product.name.trim() === "") {  
    return { valid: false, error: "Название обязательно" };  
  }  
  if (typeof product.price !== "number" || product.price <= 0) {  
    return { valid: false, error: "Цена должна быть положительным числом" };  
  }  
  const allowed = ["electronics", "clothing", "food"];  
  if (!allowed.includes(product.category)) {  
    return { valid: false, error: "Недопустимая категория" };  
  }  
  return { valid: true };  
}
```

- ❗ Функция возвращает объект с флагом `valid` и описанием ошибки — это удобный паттерн для обработки результатов валидации.

Влияние валидации на работу программы

Фильтрация

Некорректные записи отсеиваются до обработки — список содержит только валидные товары

Обработка

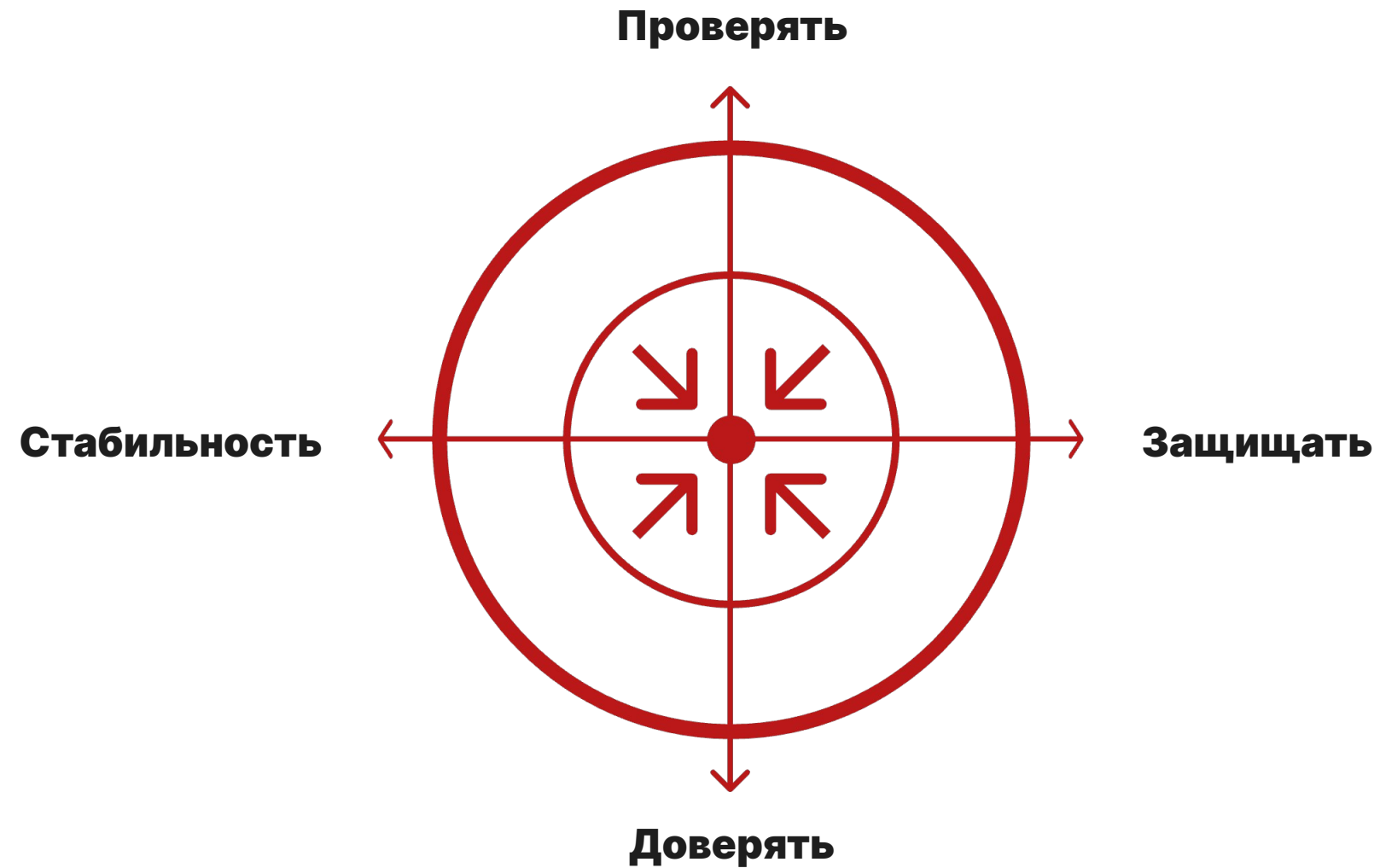
Вычисления, сортировка и группировка работают без сбоев на чистых данных

Отображение

Пользователь видит корректный интерфейс без пустых полей и неожиданных значений



Итог занятия



Валидация данных — фундаментальный навык разработчика. Она защищает программу от непредсказуемых ошибок и гарантирует, что бизнес-логика работает на надёжных, проверенных данных.